UNIT 2:

3. An expression-processing application receives an arithmetic expression in infix form. Write a C program using a stack to convert it to postfix form while correctly handling parentheses and operator precedence for +, -, *, / and ^. Test the program using an expression containing multiple operators and parentheses.

Source code :-

```
#include <stdio.h>

#include <ctype.h>

#include <string.h>

char stack[100];

int top = -1;

void push(char c){

   stack[++top] = c;

}

char pop(){

   return stack[top--];

}

int precedence(char c)

{

   if (c == '^')

      return 3;

   if (c == '*' || c == '/')

      return 2;

   if (c == '+' || c == '-')

      return 1;

   return 0;

}

int main()

{

   char infix[100], postfix[100];

   int i, j = 0;

   char c;
```

```
printf("Enter infix expression: ");
scanf("%s", infix);
for (i = 0; i < strlen(infix); i++)
{
   c = infix[i];
   if (isalnum(c))
   {
      postfix[j++] = c;
   }
   else if (c == '(')
   {
      push(c);
   }
   else if (c == ')')
   {
      while (top != -1 && stack[top] != '(')
      {
         postfix[j++] = pop();
      }
      if (top != -1)
         pop();
   }
   else
   {
      while (top != -1 &&
           stack[top] != '(' &&
           (precedence(stack[top]) > precedence(c) ||
           (precedence(stack[top]) == precedence(c) && c != '^')))
      {
         postfix[j++] = pop;
```

```
}
  push(c);
      }
   }
   while (top != -1)
   {
      postfix[j++] = pop();
   }
   postfix[j] = '\0';
   printf("Postfix expression: %s\n", postfix);
   return 0;
}
```

Output :-

```
Enter infix expression: a+b*(c-d)^e
Postfix expression: abcd-e^*+
```